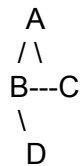


Module 9: Graph Data Structures

- Graph Representation
- Breadth First Search (BFS)
- Depth First Search (DFS)
- Shortest Path Concepts
- Introduction to Dijkstra's Algorithm
- Real-world Applications of Graphs

A **Graph** is a non-linear data structure consisting of a collection of **vertices (nodes)** connected by **edges (links)**.

$G = (V, E)$ **vertex** is a point inside graph **Edge** is connection between two vertices



Degree is no of edges connected to a vertex

Path is a sequence of connected vertices

A, B, C, D

(A, B)
(A, C)
(B, C)
(B, D)

Think about Google Maps.

Cities → Vertices

Roads → Edges

Finding shortest route

↓

Graph Problem

Why Do We Need Graphs?

Graphs are used whenever relationships exist.

Examples

- Facebook Friends
- Instagram Followers
- LinkedIn Connections
- Airline Routes
- Computer Networks
- GPS Navigation
- Power Grid
- Internet Routing

Types of graphs

1. Undirected Graph Edges have no direction. $A \leftrightarrow B$	2. Directed Graph (Digraph) Edges have direction. $A \rightarrow B$ Instagram Follow A follows B Does not imply B follows A
3. Weighted Graph Each edge has a weight. $A \text{ --5-- } B$ Weight may represent 1. Distance 2. Cost 3. Time Delhi -----250 km----- Agra	4. Unweighted Graph Every edge has equal cost.

5. Cyclic Graph

Contains cycles.

6. Acyclic Graph

No cycles.

Example Tree

How to represent Graph in C

1. Adjacency Matrix	2. Adjacency List
A 2D array storing connections. A B C D A 0 1 1 0 B 1 0 1 1 C 1 1 0 0 D 0 1 0 0	Stores neighbors of every vertex. A → B → C B → A → C → D C → A → B D → B
Advantages • Simple • Easy to implement • Fast edge lookup ○ $O(1)$	Advantages 1. Less memory 2. Space a. $O(V+E)$
Consumes more memory Requires V^2 space.	Disadvantages Searching edge takes longer.

Use Cases

1. Small graphs
2. Dense graphs
3. Network simulations

Best Used For

1. Social networks
2. Road maps
3. Internet routing
4. Sparse graphs

Feature	BFS	DFS
Data Structure	Queue	Stack
Finds Shortest Path	Yes (Unweighted)	No
Memory	More	Less
Traversal	Level Wise	Depth Wise
Backtracking	No	Yes
Applications	GPS, Networking	AI, Maze, Compiler

BFS visits all neighboring nodes first before moving to the next level.

Uses

Queue (FIFO)

DFS explores one branch completely before backtracking.

Uses

Stack

(or Recursion)

<pre> A / \ B C D /\ E F </pre>	<pre> A / \ B C D /\ E F </pre>
A B C D E F Result A B C D E F	A B E F C D
Algorithm 1. Start from source node 2. Mark visited 3. Push into Queue 4. Remove from Queue 5. Visit neighbors 6. Repeat Time & Space Complexity $O(V+E)$ & $O(V)$	Algorithm 1. Visit node 2. Mark visited 3. Visit first neighbor 4. Continue 5. Backtrack 6. Visit remaining neighbors Time & Space Complexity $O(V+E)$ & $O(V)$
Applications 1. Shortest path in unweighted graph 2. GPS 3. Social Networks 4. Web Crawling 5. Flood Fill 6. Level Order Traversal 7. Peer-to-Peer Networks	Applications 1. Maze Solving 2. Sudoku 3. Topological Sorting 4. Cycle Detection 5. Connected Components 6. Path Finding 7. AI Game Trees

--	--

Why Shortest route calculation imp ?

Shortest path means

Finding the minimum cost path between two vertices.

Cost may represent

1. Distance
2. Time
3. Money
4. Fuel

A---2---B

| |

5 1

| |

C---3---D

Shortest path: A → B → D

Cost : 2+1=3

How to calculate the shortest route in Graph data ?

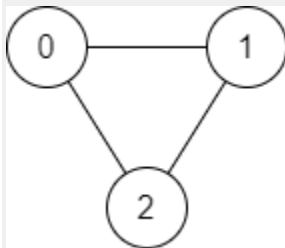
1971. Find if Path Exists in Graph

There is a **bi-directional** graph with n vertices, where each vertex is labeled from 0 to $n - 1$ (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex u_i and vertex v_i . Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

Given `edges` and the integers `n`, `source`, and `destination`, return `true` *if there is a valid path from source to destination*, or `false` *otherwise*.

Example 1:



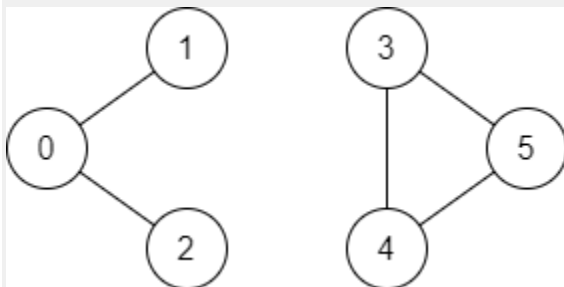
Input: `n = 3`, `edges = [[0,1],[1,2],[2,0]]`, `source = 0`, `destination = 2`

Output: `true`

Explanation: There are two paths from vertex 0 to vertex 2:

- `0 → 1 → 2`
- `0 → 2`

Example 2:



Input: `n = 6`, `edges = [[0,1],[0,2],[3,5],[5,4],[4,3]]`, `source = 0`, `destination = 5`

Output: `false`

Explanation: There is no path from vertex 0 to vertex 5.

Steps for above scenario will be :

Create adjacency list

Create visited array

Create Queue

Push source

Mark source visited

while queue not empty

 current = queue.front()

 pop queue

if current == destination

 return true

 for each neighbor

 if not visited

 visited

 push into queue

return false

Current Vertex



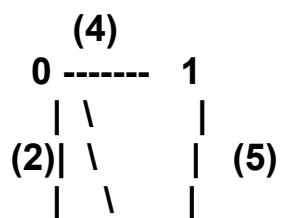
Find Neighbors



Visit First Neighbor



Repeat



| (1)\ |
2 ----- 3
(3)

Rule of Dijkstra

At every step:

1. Pick the unvisited vertex with the smallest distance.
2. Mark it as visited.
3. Update (relax) the distances of its neighbours.